ASSIGNMENT - TECH HAUS
Submitted by: Chetna Madaan

Q1.)

```
import java.util.Scanner;

/*** Program to determine how many digits are present in a given integer value. */

public class DigitLengthCalculator {

    /**
     * Calculates digit length of an integer.
     * Negative sign is ignored while counting.
     * @param value integer provided by the user
     * @return total digits present in the number
     */
    public static int calculateDigitLength(int value) {

        long normalizedValue = value;

        // Convert negative values to positive
        if (normalizedValue < 0) {
            normalizedValue = -normalizedValue;
        }

        int totalDigits = 0;

        // Zero itself contains one digit
        do {
            totalDigits++;
            normalizedValue = normalizedValue / 10;
        } while (normalizedValue != 0);

        return totalDigits;
    }

    public static void main(String[] args) {

        Scanner inputReader = new Scanner(System.in);

        System.out.print("Enter an integer value: ");
```

```
        if (!inputReader.hasNextInt()) {
        System.out.println("Invalid integer input.");
        inputReader.close();
        return;
    }

    int enteredNumber = inputReader.nextInt();


        int digitLength = calculateDigitLength(enteredNumber);

        System.out.println("Digit count = " + digitLength);

        inputReader.close();
    }
}
```

Q2)

```
import java.util.Scanner;

/**
 * Program to calculate the original product
 * price before tax was applied.
 */
public class GrossPriceCalculator {

    /**
     * Computes gross price using net price
     * and tax percentage.
     *
     * Formula:
     * gross price = net price / (1 + tax rate)
     *
     * @param finalAmount amount including tax
     * @param applicableTax tax percentage in decimal form
     * @return original amount before tax
     */
    public static double calculateGrossPrice(
            double finalAmount,
            double applicableTax) {
```

```
    if (finalAmount < 0) {
        throw new IllegalArgumentException(
                "Net price cannot be negative."
        );
    }

    if (applicableTax < 0) {
        throw new IllegalArgumentException(
                "Tax rate cannot be negative."
        );
    }

    double taxMultiplier = 1 + applicableTax;

    return finalAmount / taxMultiplier;
}

public static void main(String[] args) {

    Scanner inputScanner = new Scanner(System.in);

    System.out.print(
            "Enter net price (including tax): "
    );

    // Validate numeric net price input
    if (!inputScanner.hasNextDouble()) {

        System.out.println(
                "Invalid net price entered."
        );

        inputScanner.close();
        return;
    }

    double enteredNetPrice =
            inputScanner.nextDouble();

    System.out.print(
            "Enter tax rate (example: 0.18): "
```

```
        );

        // Validate numeric tax rate input
        if (!inputScanner.hasNextDouble()) {

            System.out.println(
                "Invalid tax rate entered."
            );

            inputScanner.close();
            return;
        }

        double enteredTaxRate =
            inputScanner.nextDouble();

        try {

            double grossAmount =
                calculateGrossPrice(
                    enteredNetPrice,
                    enteredTaxRate
                );

            System.out.printf(
                "Gross price before tax = %.2f%n",
                grossAmount
            );

        } catch (IllegalArgumentException exception) {

            System.out.println(
                exception.getMessage()
            );
        }

        inputScanner.close();
    }
}
```

Q3.)

```
import java.util.Scanner;

/**
 * Program to determine the greater value
 * between two numeric inputs.
 */
public class MaximumValueFinder {

    /**
     * Returns the larger value among two numbers.
     *
     * @param firstValue first numeric value
     * @param secondValue second numeric value
     * @return greater value between both inputs
     */
    public static double getMaximumValue(
            double firstValue,
            double secondValue) {

        if (firstValue >= secondValue) {
            return firstValue;
        }

        return secondValue;
    }

    public static void main(String[] args) {

        Scanner valueReader = new Scanner(System.in);

        System.out.print("Enter first number: ");

        // Validate first numeric input
        if (!valueReader.hasNextDouble()) {

            System.out.println(
                    "Invalid first number entered."
            );

            valueReader.close();
```

```
        return;
    }

    double initialNumber =
            valueReader.nextDouble();

    System.out.print("Enter second number: ");

    // Validate second numeric input
    if (!valueReader.hasNextDouble()) {

        System.out.println(
                "Invalid second number entered."
        );

        valueReader.close();
        return;
    }

    double comparedNumber =
            valueReader.nextDouble();

    double maximumValue =
            getMaximumValue(
                    initialNumber,
                    comparedNumber
            );

    System.out.println(
            "Larger value = " + maximumValue
    );

    valueReader.close();
  }
}
```

Q4.)

```
import java.util.Scanner;

/**
 * Program to determine the median value
```

```
 * from a sorted array of numbers.
 */
public class MedianValueCalculator {

    /**
     * Returns the median of a sorted numeric array.
     *
     * @param sortedValues array arranged in sorted order
     * @return median value of the array
     * @throws IllegalArgumentException
     *         if array is null or empty
     */
    public static double calculateMedian(
            double[] sortedValues) {

        if (sortedValues == null
                || sortedValues.length == 0) {

            throw new IllegalArgumentException(
                    "Array must contain at least one element."
            );
        }

        int totalElements = sortedValues.length;

        int middlePosition = totalElements / 2;

        // Median for odd-sized array
        if (totalElements % 2 != 0) {

            return sortedValues[middlePosition];
        }

        // Median for even-sized array
        double leftMiddleValue =
                sortedValues[middlePosition - 1];

        double rightMiddleValue =
                sortedValues[middlePosition];

        return (leftMiddleValue + rightMiddleValue)
```

```
        / 2.0;
}

public static void main(String[] args) {

    Scanner inputHandler = new Scanner(System.in);

    System.out.print(
        "Enter number of elements: "
    );

    // Validate array size input
    if (!inputHandler.hasNextInt()) {

        System.out.println(
            "Invalid array size entered."
        );

        inputHandler.close();
        return;
    }

    int arraySize = inputHandler.nextInt();

    if (arraySize <= 0) {

        System.out.println(
            "Array size must be greater than zero."
        );

        inputHandler.close();
        return;
    }

    double[] enteredValues =
        new double[arraySize];

    System.out.println(
        "Enter sorted array values:"
    );
```

```
        for (int index = 0;
            index < arraySize;
            index++) {

          // Validate numeric array input
          if (!inputHandler.hasNextDouble()) {

            System.out.println(
                  "Invalid numeric value entered."
            );

            inputHandler.close();
            return;
          }

          enteredValues[index] =
                inputHandler.nextDouble();
        }

        try {

          double medianValue =
                calculateMedian(enteredValues);

          System.out.println(
                "Median value = " + medianValue
          );

        } catch (IllegalArgumentException exception) {

          System.out.println(
                exception.getMessage()
          );
        }

        inputHandler.close();
      }
    }
```

Q5.)

```
import java.util.Scanner;

/**
 * Program to calculate income tax
 * according to defined tax slabs.
 */
public class IncomeTaxCalculator {

    private static final double BASIC_EXEMPTION_LIMIT =
            250000;

    private static final double SECOND_SLAB_LIMIT =
            500000;

    private static final double THIRD_SLAB_LIMIT =
            1000000;

    /**
     * Calculates payable tax amount
     * based on annual income.
     *
     * @param annualIncome total yearly income
     * @return calculated income tax
     */
    public static double calculateTaxAmount(
            double annualIncome) {

        if (annualIncome < 0) {

            throw new IllegalArgumentException(
                    "Income value cannot be negative."
            );
        }

        double payableTax = 0;

        // Tax slab between 2.5L and 5L
        if (annualIncome > BASIC_EXEMPTION_LIMIT) {

            double taxableAmountInSecondSlab =
```

```
            Math.min(
                annualIncome,
                SECOND_SLAB_LIMIT
            ) - BASIC_EXEMPTION_LIMIT;

        payableTax +=
            taxableAmountInSecondSlab * 0.10;
    }

    // Tax slab between 5L and 10L
    if (annualIncome > SECOND_SLAB_LIMIT) {

        double taxableAmountInThirdSlab =
            Math.min(
                annualIncome,
                THIRD_SLAB_LIMIT
            ) - SECOND_SLAB_LIMIT;

        payableTax +=
            taxableAmountInThirdSlab * 0.20;
    }

    // Tax slab above 10L
    if (annualIncome > THIRD_SLAB_LIMIT) {

        double taxableAmountAboveTenLakhs =
            annualIncome - THIRD_SLAB_LIMIT;

        payableTax +=
            taxableAmountAboveTenLakhs * 0.30;
    }

    return payableTax;
}

public static void main(String[] args) {

    Scanner incomeReader =
        new Scanner(System.in);

    System.out.print("Enter annual income: ");
```

```
        // Validate numeric income input
        if (!incomeReader.hasNextDouble()) {

            System.out.println(
                    "Invalid income value entered."
            );

            incomeReader.close();
            return;
        }

        double enteredIncome =
                incomeReader.nextDouble();

        try {

            double calculatedTax =
                    calculateTaxAmount(
                            enteredIncome
                    );

            System.out.printf(
                    "Calculated income tax = %.2f%n",
                    calculatedTax
            );

        } catch (IllegalArgumentException exception) {

            System.out.println(
                    exception.getMessage()
            );
        }

        incomeReader.close();
    }
}
```

Q6.)

```
import java.util.Scanner;

/**
 * Program to identify the word
 * containing the highest number of vowels.
 */
public class VowelRichWordFinder {

    /**
     * Counts vowels present in a word.
     *
     * @param inputWord word to be evaluated
     * @return total vowel count
     */
    public static int calculateVowelCount(
            String inputWord) {

        if (inputWord == null) {

            throw new IllegalArgumentException(
                    "Input word cannot be null."
            );
        }

        int vowelCount = 0;

        String normalizedWord =
                inputWord.toLowerCase();

        for (int characterIndex = 0;
             characterIndex < normalizedWord.length();
             characterIndex++) {

            char currentCharacter =
                    normalizedWord.charAt(characterIndex);

            if (currentCharacter == 'a'
                    || currentCharacter == 'e'
                    || currentCharacter == 'i'
                    || currentCharacter == 'o'
```

```
            || currentCharacter == 'u') {

        vowelCount++;
    }
  }

  return vowelCount;
}

/**
 * Returns the word containing
 * the maximum number of vowels.
 *
 * @param wordCollection array of words
 * @return word with highest vowel count
 */
public static String getWordWithMaximumVowels(
    String[] wordCollection) {

  if (wordCollection == null
        || wordCollection.length == 0) {

    throw new IllegalArgumentException(
          "Input array cannot be null or empty."
    );
  }

  String selectedWord =
        wordCollection[0];

  int highestVowelCount =
        calculateVowelCount(selectedWord);

  for (int position = 1;
      position < wordCollection.length;
      position++) {

    int currentVowelCount =
          calculateVowelCount(
                wordCollection[position]
          );
```

```
        if (currentVowelCount > highestVowelCount) {

            highestVowelCount =
                    currentVowelCount;

            selectedWord =
                    wordCollection[position];
        }
    }

    return selectedWord;
}

public static void main(String[] args) {

    Scanner textReader =
            new Scanner(System.in);

    System.out.print(
            "Enter number of words: "
    );

    // Validate word count input
    if (!textReader.hasNextInt()) {

        System.out.println(
                "Invalid word count entered."
        );

        textReader.close();
        return;
    }

    int totalWords =
            textReader.nextInt();

    textReader.nextLine();

    if (totalWords <= 0) {
```

```
    System.out.println(
            "Word count must be greater than zero."
    );

    textReader.close();
    return;
}

String[] enteredWords =
        new String[totalWords];

System.out.println("Enter words:");

for (int index = 0;
     index < totalWords;
     index++) {

    String currentWord =
            textReader.nextLine().trim();

    // Validate empty word input
    if (currentWord.isEmpty()) {

        System.out.println(
                "Word input cannot be empty."
        );

        textReader.close();
        return;
    }

    enteredWords[index] = currentWord;
}

try {

    String vowelRichWord =
            getWordWithMaximumVowels(
                    enteredWords
            );
```

```
        System.out.println(
            "Word with highest vowels = "
                + vowelRichWord
        );

    } catch (IllegalArgumentException exception) {

        System.out.println(
            exception.getMessage()
        );
    }

    textReader.close();
  }
}
```

Q7.)

```
import java.util.Scanner;

/**
 * Program to calculate compounded
 * retail inflation over a specified period.
 */
public class RetailInflationCalculator {

  /**
   * Computes compounded inflation percentage.
   * Formula:
   * (1 + inflationRate)^years - 1
   * @param yearlyInflationRate
   *       yearly inflation rate in decimal form
   * @param totalYears
   *       total number of years
   * @return compounded inflation percentage
   */
  public static double calculateCompoundedInflation(
        double yearlyInflationRate,
        int totalYears) {

    if (yearlyInflationRate < 0) {
```

```
        throw new IllegalArgumentException(
                "Inflation rate cannot be negative."
        );
    }

    if (totalYears < 0) {

        throw new IllegalArgumentException(
                "Years value cannot be negative."
        );
    }

    double growthFactor =
            Math.pow(
                    1 + yearlyInflationRate,
                    totalYears
            );

    double compoundedInflation =
            growthFactor - 1;

    return compoundedInflation * 100;
}

public static void main(String[] args) {

    Scanner inflationReader =
            new Scanner(System.in);

    System.out.print(
            "Enter yearly inflation rate "
                    + "(example: 0.08): "
    );

    // Validate inflation rate input
    if (!inflationReader.hasNextDouble()) {

        System.out.println(
                "Invalid inflation rate entered."
        );
```

```
    inflationReader.close();
    return;
}

double enteredRate =
        inflationReader.nextDouble();

System.out.print(
        "Enter number of years: "
);

// Validate years input
if (!inflationReader.hasNextInt()) {

    System.out.println(
            "Invalid year value entered."
    );

    inflationReader.close();
    return;
}

int enteredYears =
        inflationReader.nextInt();

try {

    double inflationPercentage =
            calculateCompoundedInflation(
                    enteredRate,
                    enteredYears
            );

    System.out.printf(
            "Compounded inflation = %.2f%%%n",
            inflationPercentage
    );

} catch (IllegalArgumentException exception) {

    System.out.println(
```

```
                exception.getMessage()
            );
        }

        inflationReader.close();
    }
}
```

Q8.)

```
import java.util.Scanner;

/**
 * Program to convert a total number
 * of seconds into days, hours,
 * minutes, and seconds.
 */
public class TimeDurationFormatter {

    private static final long SECONDS_IN_A_MINUTE =
            60;

    private static final long SECONDS_IN_AN_HOUR =
            3600;

    private static final long SECONDS_IN_A_DAY =
            86400;

    /**
     * Converts seconds into
     * formatted time duration.
     * @param inputSeconds
     *        total seconds provided by user
     * @return formatted duration string
     */
    public static String formatTimeDuration(
            long inputSeconds) {

        if (inputSeconds < 0) {

            throw new IllegalArgumentException(
                    "Seconds value cannot be negative."
```

```
        );
    }

    long calculatedDays =
            inputSeconds / SECONDS_IN_A_DAY;

    long remainingSecondsAfterDays =
            inputSeconds % SECONDS_IN_A_DAY;

    long calculatedHours =
            remainingSecondsAfterDays
                    / SECONDS_IN_AN_HOUR;

    long remainingSecondsAfterHours =
            remainingSecondsAfterDays
                    % SECONDS_IN_AN_HOUR;

    long calculatedMinutes =
            remainingSecondsAfterHours
                    / SECONDS_IN_A_MINUTE;

    long calculatedSeconds =
            remainingSecondsAfterHours
                    % SECONDS_IN_A_MINUTE;

    return calculatedDays + " Day(s) "
            + calculatedHours + " Hour(s) "
            + calculatedMinutes + " Minute(s) "
            + calculatedSeconds + " Second(s)";
}

public static void main(String[] args) {

    Scanner durationReader =
            new Scanner(System.in);

    System.out.print(
            "Enter total seconds: "
    );

    // Validate seconds input
```

```
        if (!durationReader.hasNextLong()) {

            System.out.println(
                    "Invalid seconds value entered."
            );

            durationReader.close();
            return;
        }

        long enteredSeconds =
                durationReader.nextLong();

        try {

            String formattedDuration =
                    formatTimeDuration(
                            enteredSeconds
                    );

            System.out.println(
                    "Converted duration = "
                            + formattedDuration
            );

        } catch (IllegalArgumentException exception) {

            System.out.println(
                    exception.getMessage()
            );
        }

        durationReader.close();
    }
}
```

Q9.)

```
import java.util.Scanner;

/**
 * Program to determine whether
```

```
 * a two-digit number satisfies
 * the special integer condition.
 */
public class SpecialNumberValidator {

    /**
     * Checks whether a number
     * is a special integer.
     * Condition:
     * number = firstDigit
     *        + secondDigit
     *        + (firstDigit * secondDigit)
     * @param inputNumber
     *        number to be evaluated
     * @return true if number is special,
     *         otherwise false
     */
    public static boolean checkSpecialNumber(
            int inputNumber) {

        // Validate two-digit number range
        if (inputNumber < 10
                || inputNumber > 99) {

            return false;
        }

        int firstDigit =
                inputNumber / 10;

        int secondDigit =
                inputNumber % 10;

        int calculatedValue =
                firstDigit
                + secondDigit
                + (firstDigit * secondDigit);

        return calculatedValue == inputNumber;
    }
```

```
public static void main(String[] args) {

    Scanner numberReader =
            new Scanner(System.in);

    System.out.print(
            "Enter a two-digit number: "
    );

    // Validate numeric input
    if (!numberReader.hasNextInt()) {

        System.out.println(
                "Invalid numeric value entered."
        );

        numberReader.close();
        return;
    }

    int enteredValue =
            numberReader.nextInt();

    try {

        boolean specialStatus =
                checkSpecialNumber(
                        enteredValue
                );

        if (specialStatus) {

            System.out.println(
                    enteredValue
                            + " is a special integer."
            );

        } else {

            System.out.println(
                    enteredValue
```

```
                    + " is not a special integer."
            );
        }

    } catch (IllegalArgumentException exception) {

        System.out.println(
            exception.getMessage()
        );
    }

    numberReader.close();
  }
}
```

Q10.)

```
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
import java.util.Scanner;

/**
 * Represents a railway booking record.
 */
class BookingRecord {

  private String pnrCode;
  private String passengerName;
  private String trainIdentifier;
  private String boardingStation;
  private String destinationStation;
  private LocalDate journeyDate;
  private String coachCategory;
  private double ticketFare;

  public BookingRecord(
      String pnrCode,
```

```
        String passengerName,
        String trainIdentifier,
        String boardingStation,
        String destinationStation,
        LocalDate journeyDate,
        String coachCategory,
        double ticketFare) {

    this.pnrCode = pnrCode;
    this.passengerName = passengerName;
    this.trainIdentifier = trainIdentifier;
    this.boardingStation = boardingStation;
    this.destinationStation = destinationStation;
    this.journeyDate = journeyDate;
    this.coachCategory = coachCategory;
    this.ticketFare = ticketFare;
  }

  public String getPnrCode() {
    return pnrCode;
  }

  public String getPassengerName() {
    return passengerName;
  }

  public LocalDate getJourneyDate() {
    return journeyDate;
  }

  @Override
  public String toString() {

    return String.format(
        "PNR: %-10s | Passenger: %-18s | Train: %-8s | %s -> %s | Date: %s | Class: %-10s |
Fare: %.2f",
        pnrCode,
        passengerName,
        trainIdentifier,
        boardingStation,
        destinationStation,
```

```
            journeyDate,
            coachCategory,
            ticketFare
        );
    }
}

/**
 * Railway booking management application.
 */
public class RailwayBookingManager {

    private static final Scanner INPUT_SCANNER =
            new Scanner(System.in);

    private static final DateTimeFormatter DATE_FORMATTER =
            DateTimeFormatter.ofPattern("dd-MM-yyyy");

    private static final List<BookingRecord> BOOKING_DATABASE =
            new ArrayList<>();

    public static void main(String[] args) {

        initializeSampleBookings();

        boolean applicationRunning = true;

        while (applicationRunning) {

            displayMenu();

            String selectedOption =
                    INPUT_SCANNER.nextLine().trim();

            switch (selectedOption) {

                case "1":
                    displayBookingsSortedByDate();
                    break;

                case "2":
```

```
            registerBooking();
            break;

        case "3":
            findBookingByPNR();
            break;

        case "4":
            findBookingByPassengerName();
            break;

        case "5":

            applicationRunning = false;

            System.out.println(
                    "Booking system closed successfully."
            );

            break;

        default:

            System.out.println(
                    "Invalid menu option selected."
            );
        }
    }

    INPUT_SCANNER.close();
}

/**
 * Loads sample booking data.
 */
private static void initializeSampleBookings() {

    BOOKING_DATABASE.add(
        new BookingRecord(
            "PNR101",
            "Rohan Malhotra",
```

```
                "12045",
                "Delhi",
                "Mumbai",
                LocalDate.of(2026, 6, 18),
                "Third AC",
                1850.00
            )
        );

        BOOKING_DATABASE.add(
            new BookingRecord(
                "PNR102",
                "Neha Sharma",
                "22411",
                "Pune",
                "Jaipur",
                LocalDate.of(2026, 5, 25),
                "Sleeper",
                920.00
            )
        );

        BOOKING_DATABASE.add(
            new BookingRecord(
                "PNR103",
                "Aman Verma",
                "19322",
                "Chennai",
                "Kolkata",
                LocalDate.of(2026, 7, 5),
                "Second AC",
                2600.00
            )
        );
    }

    /**
     * Displays application menu.
     */
    private static void displayMenu() {
```

```
    System.out.println(
            "\n===== RAILWAY BOOKING MENU =====")
    );

    System.out.println("1. View all bookings");
    System.out.println("2. Add booking");
    System.out.println("3. Search booking by PNR");
    System.out.println("4. Search booking by passenger name");
    System.out.println("5. Exit");

    System.out.print("Choose an option: ");
}

/**
 * Displays bookings sorted
 * by latest journey date.
 */
private static void displayBookingsSortedByDate() {

    if (BOOKING_DATABASE.isEmpty()) {

        System.out.println(
                "No booking records available."
        );

        return;
    }

    List<BookingRecord> sortedBookings =
            new ArrayList<>(BOOKING_DATABASE);

    Collections.sort(
            sortedBookings,
            Comparator.comparing(
                    BookingRecord::getJourneyDate
            ).reversed()
    );

    for (BookingRecord booking : sortedBookings) {

        System.out.println(booking);
```

```
    }
}

/**
 * Adds a new booking entry.
 */
private static void registerBooking() {

    System.out.print("Enter PNR code: ");

    String enteredPNR =
            INPUT_SCANNER.nextLine().trim();

    // Validate empty PNR
    if (enteredPNR.isEmpty()) {

        System.out.println(
                "PNR code cannot be empty."
        );

        return;
    }

    // Validate duplicate PNR
    if (isDuplicatePNR(enteredPNR)) {

        System.out.println(
                "Booking with this PNR already exists."
        );

        return;
    }

    System.out.print("Enter passenger name: ");

    String enteredPassengerName =
            INPUT_SCANNER.nextLine().trim();

    // Validate empty passenger name
    if (enteredPassengerName.isEmpty()) {
```

```
    System.out.println(
            "Passenger name cannot be empty."
    );

    return;
}

System.out.print("Enter train number: ");

String enteredTrainNumber =
        INPUT_SCANNER.nextLine().trim();

// Validate empty train number
if (enteredTrainNumber.isEmpty()) {

    System.out.println(
            "Train number cannot be empty."
    );

    return;
}

System.out.print("Enter boarding station: ");

String enteredBoardingStation =
        INPUT_SCANNER.nextLine().trim();

// Validate empty boarding station
if (enteredBoardingStation.isEmpty()) {

    System.out.println(
            "Boarding station cannot be empty."
    );

    return;
}

System.out.print("Enter destination station: ");

String enteredDestinationStation =
        INPUT_SCANNER.nextLine().trim();
```

```
// Validate empty destination station
if (enteredDestinationStation.isEmpty()) {

    System.out.println(
            "Destination station cannot be empty."
    );

    return;
}

// Validate same source and destination
if (enteredBoardingStation.equalsIgnoreCase(
        enteredDestinationStation)) {

    System.out.println(
            "Origin and destination cannot be same."
    );

    return;
}

LocalDate enteredJourneyDate = null;

while (enteredJourneyDate == null) {

    System.out.print(
            "Enter journey date (dd-MM-yyyy): "
    );

    try {

        enteredJourneyDate =
                LocalDate.parse(
                        INPUT_SCANNER.nextLine().trim(),
                        DATE_FORMATTER
                );

    } catch (DateTimeParseException exception) {

        System.out.println(
```

```
            "Invalid date format entered."
        );
    }
}

System.out.print("Enter coach category: ");

String enteredCoachCategory =
        INPUT_SCANNER.nextLine().trim();

// Validate empty coach category
if (enteredCoachCategory.isEmpty()) {

    System.out.println(
            "Coach category cannot be empty."
    );

    return;
}

double enteredFare = 0;

while (enteredFare <= 0) {

    System.out.print("Enter fare amount: ");

    String fareInput =
            INPUT_SCANNER.nextLine().trim();

    try {

        enteredFare =
                Double.parseDouble(fareInput);

        if (enteredFare <= 0) {

            System.out.println(
                    "Fare must be greater than zero."
            );
        }
```

```
        } catch (NumberFormatException exception) {

            System.out.println(
                "Invalid fare value entered."
            );
        }
    }

    BookingRecord newBooking =
            new BookingRecord(
                enteredPNR,
                enteredPassengerName,
                enteredTrainNumber,
                enteredBoardingStation,
                enteredDestinationStation,
                enteredJourneyDate,
                enteredCoachCategory,
                enteredFare
            );

    BOOKING_DATABASE.add(newBooking);

    System.out.println(
            "Booking record added successfully."
    );
}

/**
 * Searches booking using exact PNR match.
 */
private static void findBookingByPNR() {

    System.out.print(
            "Enter PNR code to search: "
    );

    String searchedPNR =
            INPUT_SCANNER.nextLine().trim();

    if (searchedPNR.isEmpty()) {
```

```
        System.out.println(
                "PNR search value cannot be empty."
        );

        return;
    }

    for (BookingRecord booking : BOOKING_DATABASE) {

        if (booking.getPnrCode().equals(searchedPNR)) {

            System.out.println(booking);
            return;
        }
    }

    System.out.println(
            "No booking found for provided PNR."
    );
}

/**
 * Searches bookings using passenger name.
 */
private static void findBookingByPassengerName() {

    System.out.print(
            "Enter passenger name keyword: "
    );

    String searchKeyword =
            INPUT_SCANNER.nextLine()
                    .trim()
                    .toLowerCase();

    if (searchKeyword.isEmpty()) {

        System.out.println(
                "Passenger search keyword cannot be empty."
        );
```

```
        return;
    }

    boolean bookingFound = false;

    for (BookingRecord booking : BOOKING_DATABASE) {

        String normalizedPassengerName =
                booking.getPassengerName()
                        .toLowerCase();

        if (normalizedPassengerName.contains(
                searchKeyword)) {

            System.out.println(booking);

            bookingFound = true;
        }
    }

    if (!bookingFound) {

        System.out.println(
                "No passenger records matched."
        );
    }
}

/**
 * Checks whether a PNR already exists.
 *
 * @param enteredPNR
 *       user provided PNR
 *
 * @return true if duplicate exists
 */
private static boolean isDuplicatePNR(
        String enteredPNR) {

    for (BookingRecord booking : BOOKING_DATABASE) {
```

```
            if (booking.getPnrCode().equals(
                    enteredPNR)) {

                return true;
            }
        }

        return false;
    }
}
```